



Computer System Overview

Dept. of Computer Science
Hanyang University





Basic Elements

- Processor (CPU)
- Main Memory
 - volatile
 - referred to as real memory or primary memory
- System bus
 - communication among processors, memory, and I/O modules
- I/O modules
 - secondary memory devices
 - communications equipment
 - terminals



Processor Registers

- Registers for 'data' I/O
 - Memory address register (MAR)
 - Specifies the address for the next read (LOAD) or write (STORE)
 - Memory buffer register (MBR)
 - Contains data that will be written to memory: STORE
 - or stores data that has been read from memory: LOAD
 - I/O address register, I/O buffer register
- Control and status registers
 - Used by processor to control operating of the processor
 - Used by privileged operating-system routines to control the execution of programs
- User-visible registers
 - Enable programmer to minimize main-memory references by optimizing register use (Referenced by machine language)



Control and Status Registers

- Program Counter (PC)
 - Contains the address of an instruction to be fetched
- Instruction Register (IR)
 - Contains the instruction most recently fetched
- Program Status Word (PSW)
 - Condition codes
 - Bits set by the processor hardware as a result of operations
 - Mainly used for conditional operations (jump in the for, while or if statement)
 - Examples
 - Positive result
 - Negative result
 - Zero
 - Overflow
 - Interrupt enable/disable
 - Supervisor/user mode

Instruction Execution

- Two steps
 - Processor reads instructions from memory: **Fetch**
 - Processor executes each instruction
- PC register holds address of the instruction to be fetched next
- PC register is incremented after each fetch

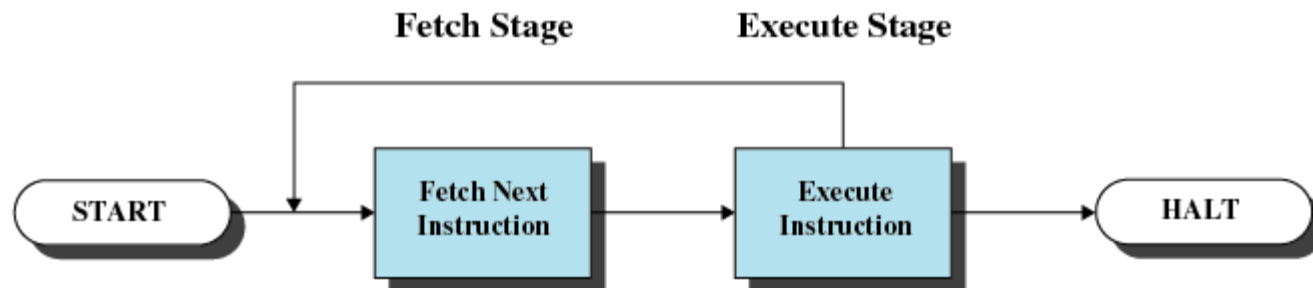


Figure 1.2 Basic Instruction Cycle



Instruction Register (IR)

- Fetched instruction is placed in the IR
- Instruction categories
 - Processor-memory
 - Transfer data between processor and memory
 - Processor-I/O
 - Data transferred to or from a peripheral device
 - Data processing
 - Arithmetic or logic operation on data
 - Control
 - Alter sequence of execution

Top-Level Components

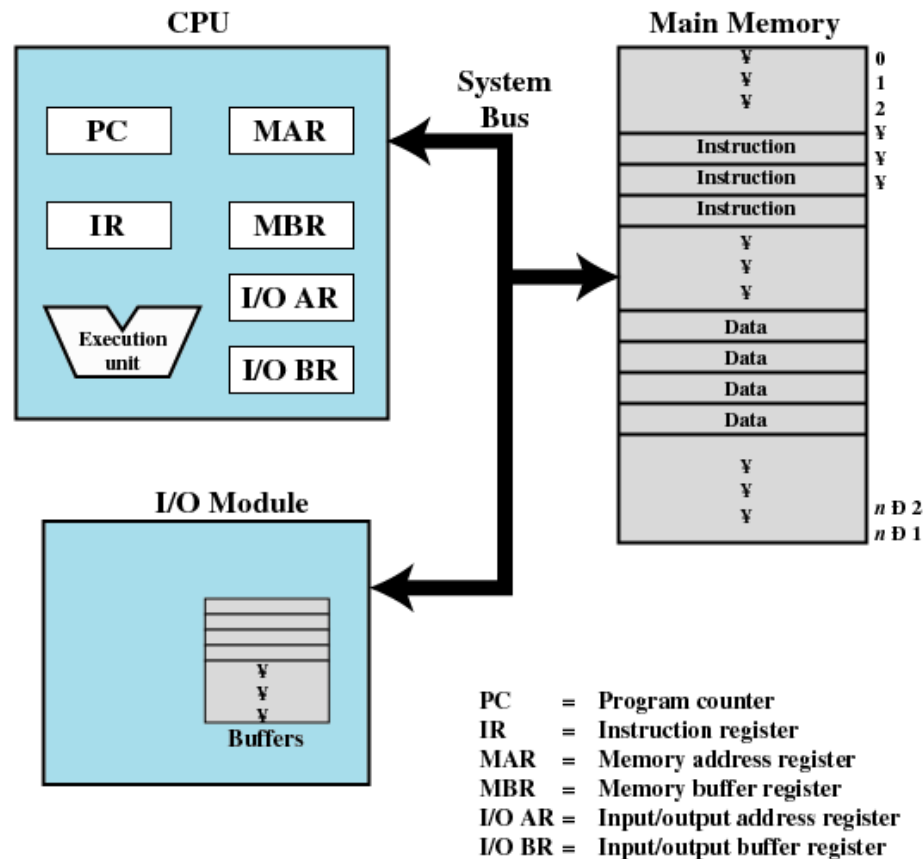
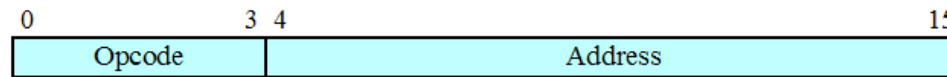


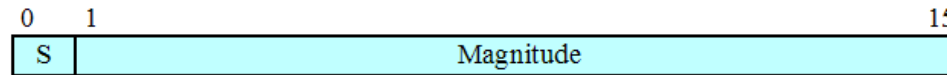
Figure 1.1 Computer Components: Top-Level View



Characteristics of a Hypothetical Machine



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction

Instruction Register (IR) = Instruction being executed

Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory

0010 = Store AC to Memory

0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 1.3 Characteristics of a Hypothetical Machine

Example of Program Execution

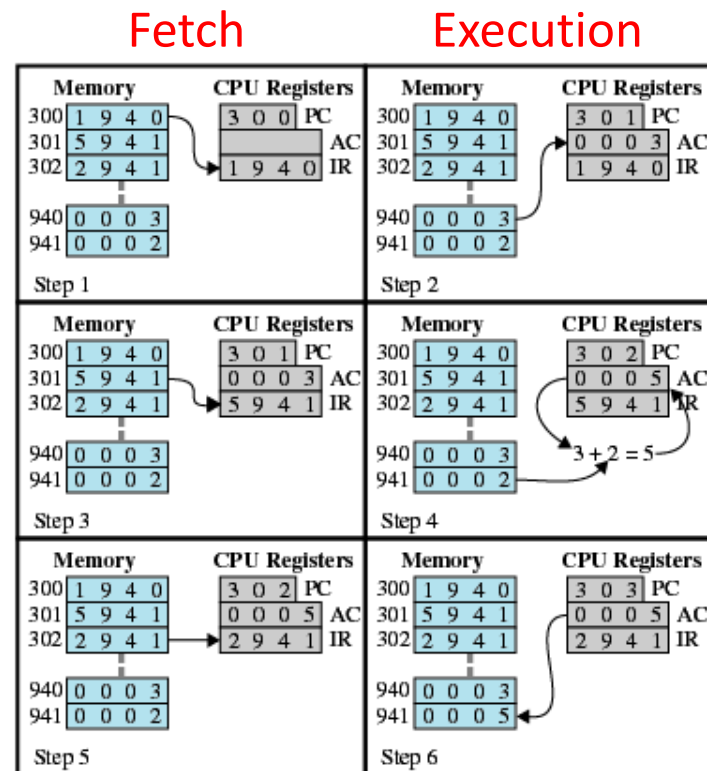


Figure 1.4 Example of Program Execution
(contents of memory and registers in hexadecimal)

Interrupt & Interrupt handler

- **Interrupt**
 - Interrupt the normal sequencing of the processor
 - Most I/O devices are slower than the processor
 - Processor must pause to wait for device
- **Interrupt handler**
 - Program to service a particular I/O device
 - Generally part of the operating system

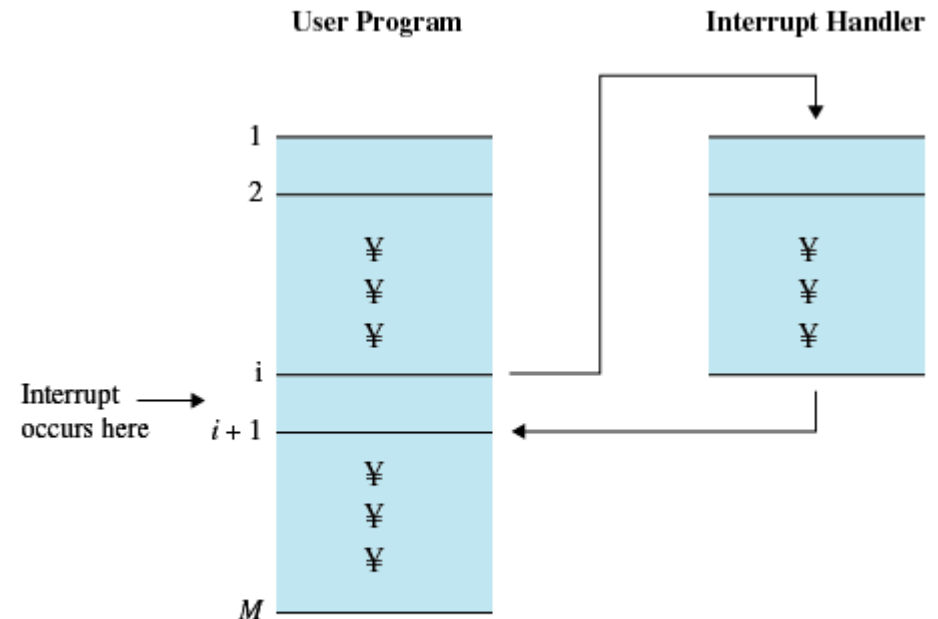


Figure 1.6 Transfer of Control via Interrupts

Interrupt Cycle

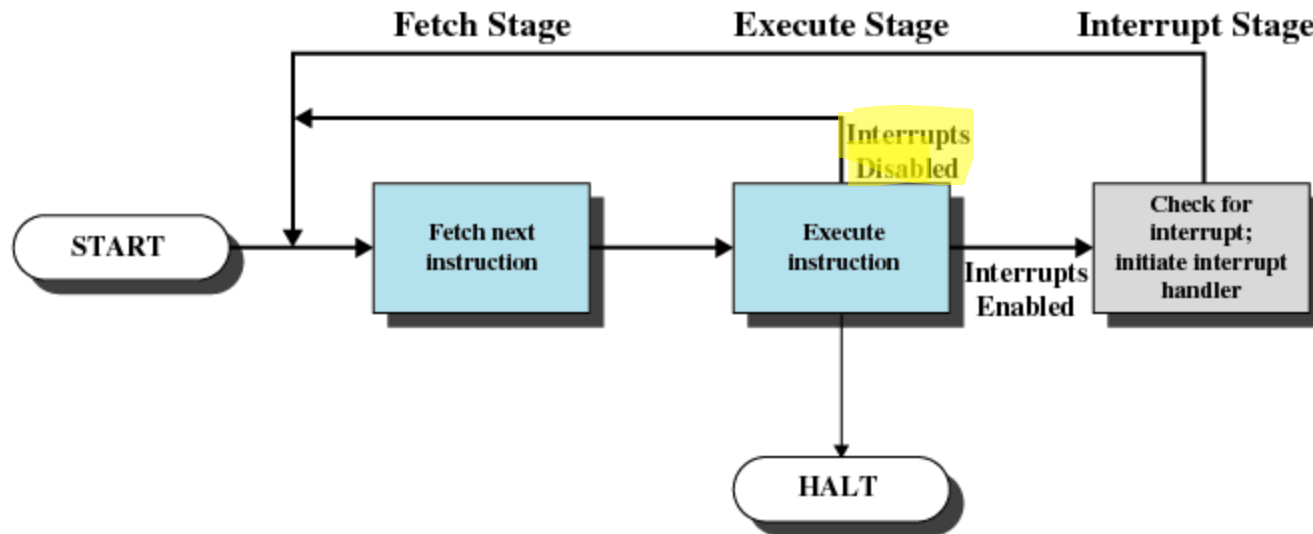


Figure 1.7 Instruction Cycle with Interrupts

Simple Interrupt Processing

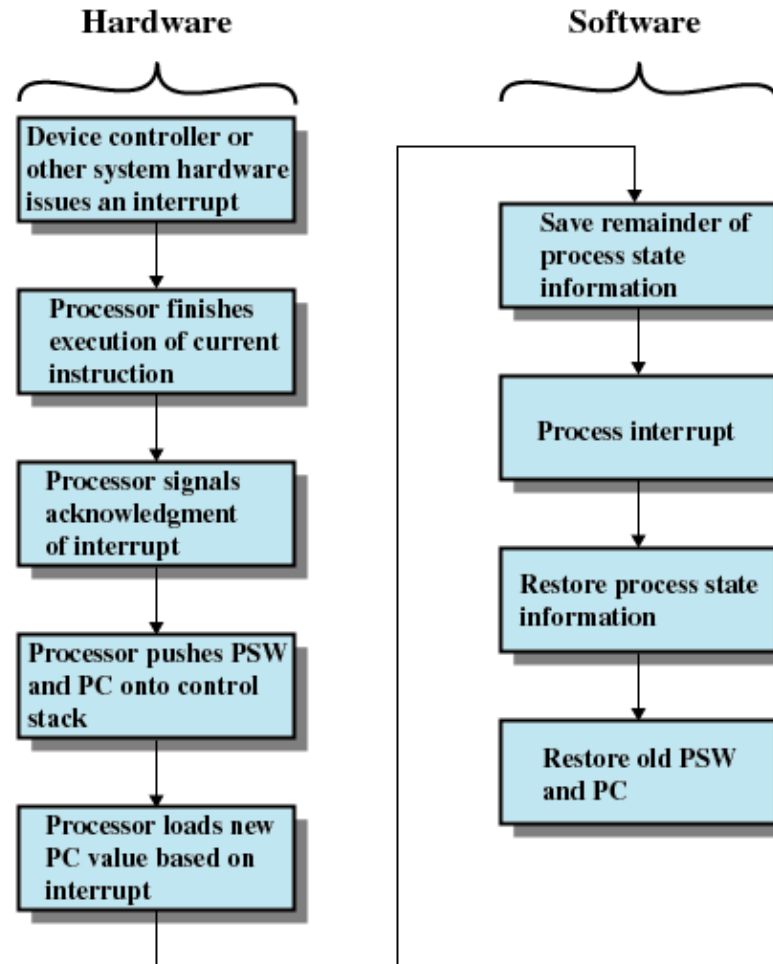
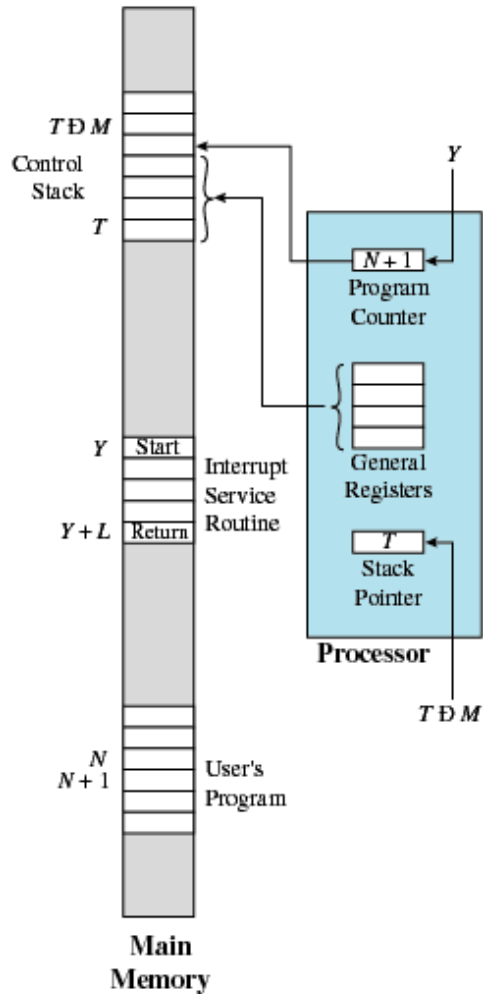
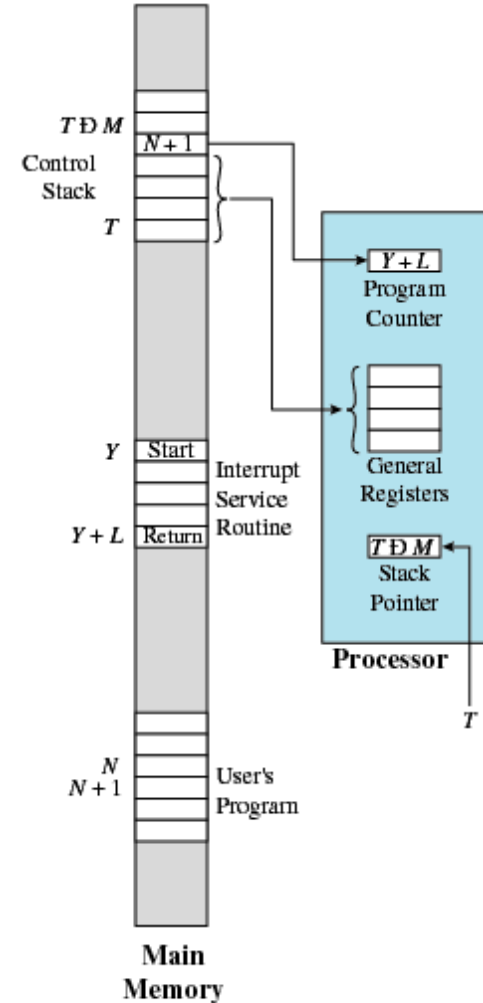
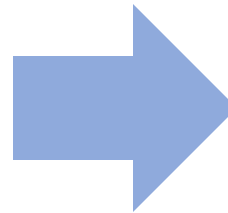


Figure 1.10 Simple Interrupt Processing

Changes in Memory and Registers for an Interrupt

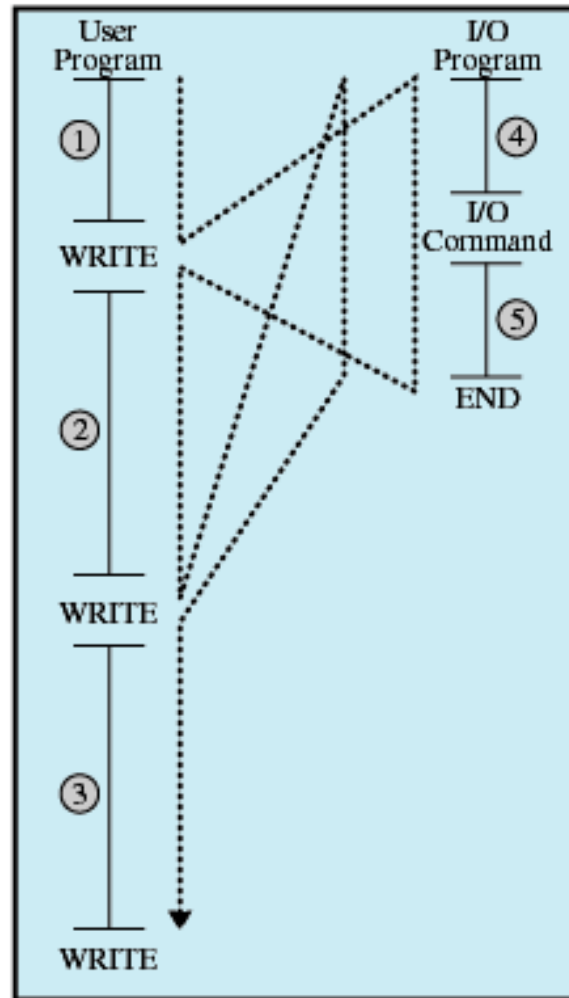


(a) Interrupt occurs after instruction at location N



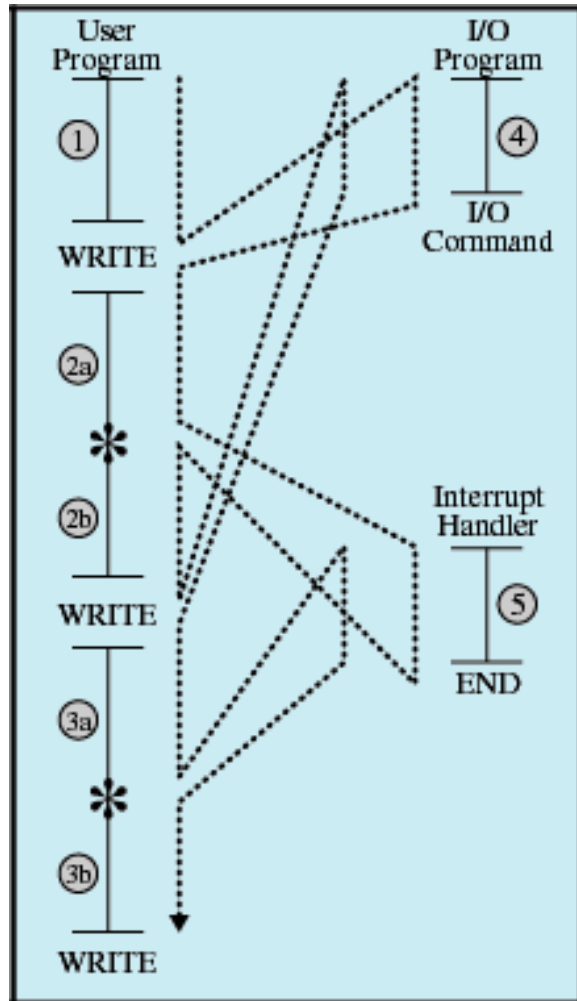
(b) Return from interrupt

Program Flow of Control without Interrupts

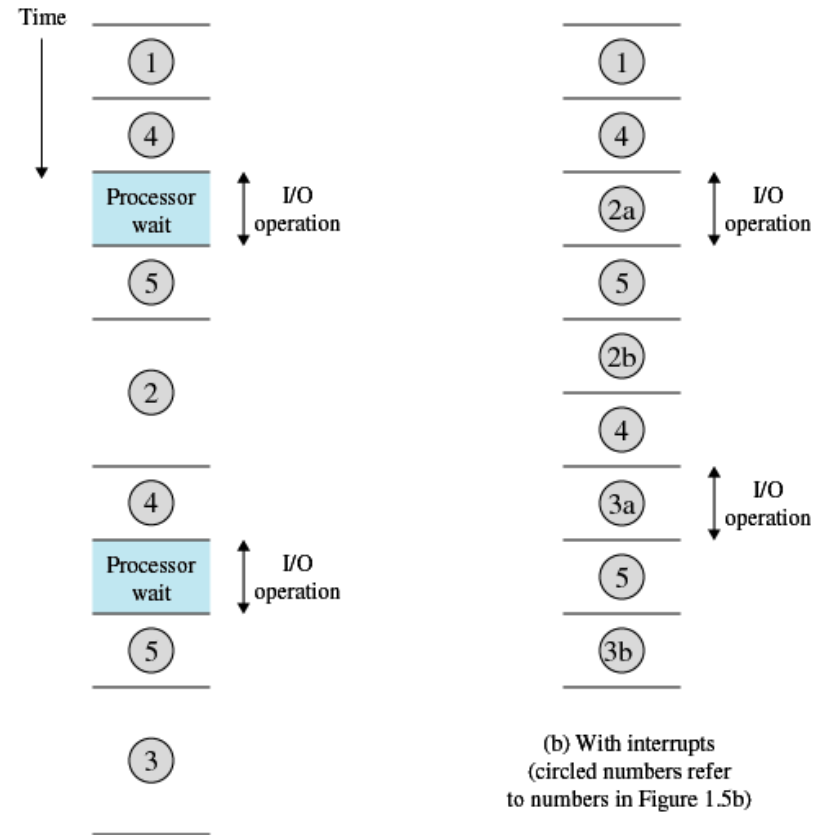


(a) No interrupts

Program Flow of Control with Interrupts: Short I/O Wait



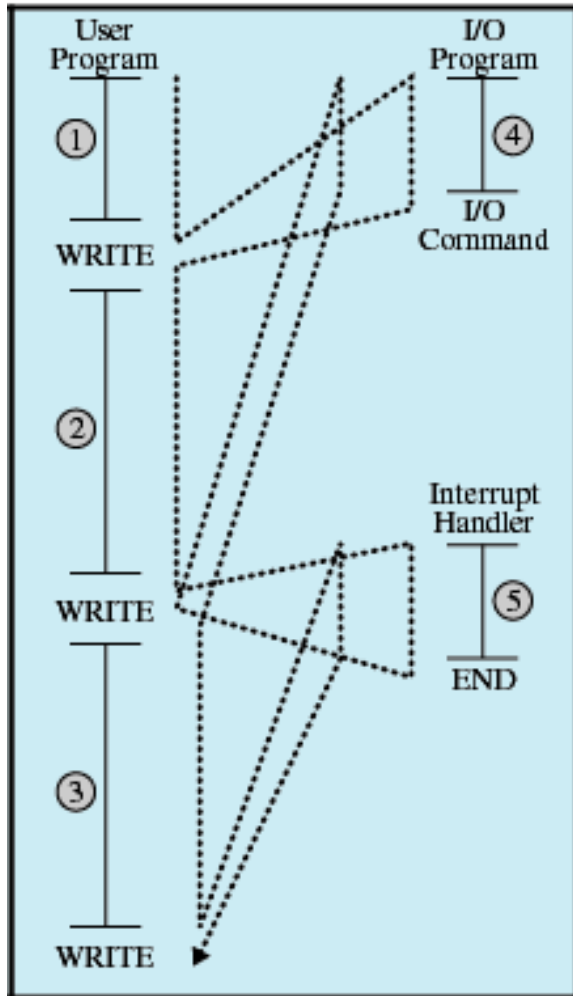
(b) Interrupts; short I/O wait



(a) Without interrupts
(circled numbers refer to numbers in Figure 1.5a)

Figure 1.8 Program Timing: Short I/O Wait

Program Flow of Control with Interrupts; Long I/O Wait



(c) Interrupts; long I/O wait

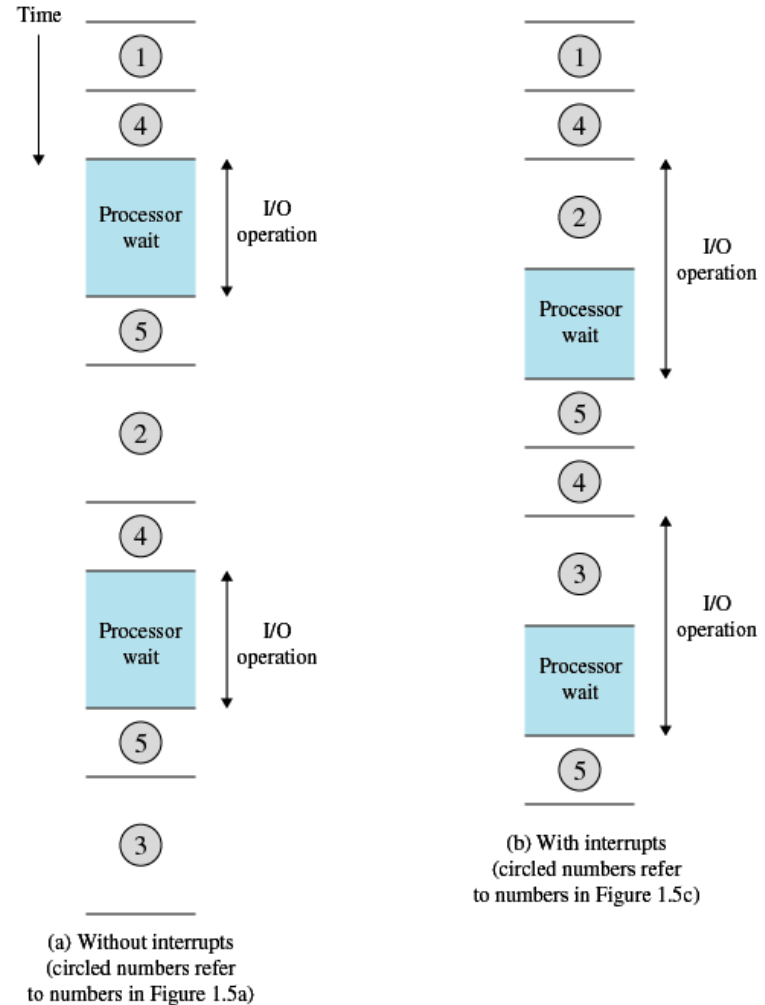


Figure 1.9 Program Timing: Long I/O Wait



Multiprogramming

- Even using interrupts, processor may not be used efficiently
 - A solution is **multiprogramming**
- Processor has more than one program to execute
- The sequence the programs are executed depends on their relative priority and whether they are waiting for I/O
- After an interrupt handler completes, control may not return to the program that was executing at the time of the interrupt

Memory Hierarchy

- Going up the hierarchy
 - Faster access time
 - Decreasing capacity
 - Greater cost per bit
- Going down the hierarchy
 - Decreasing cost per bit
 - Increasing capacity
 - Increasing access time
 - Decreasing frequency of access of the memory by the processor
 - Locality of reference

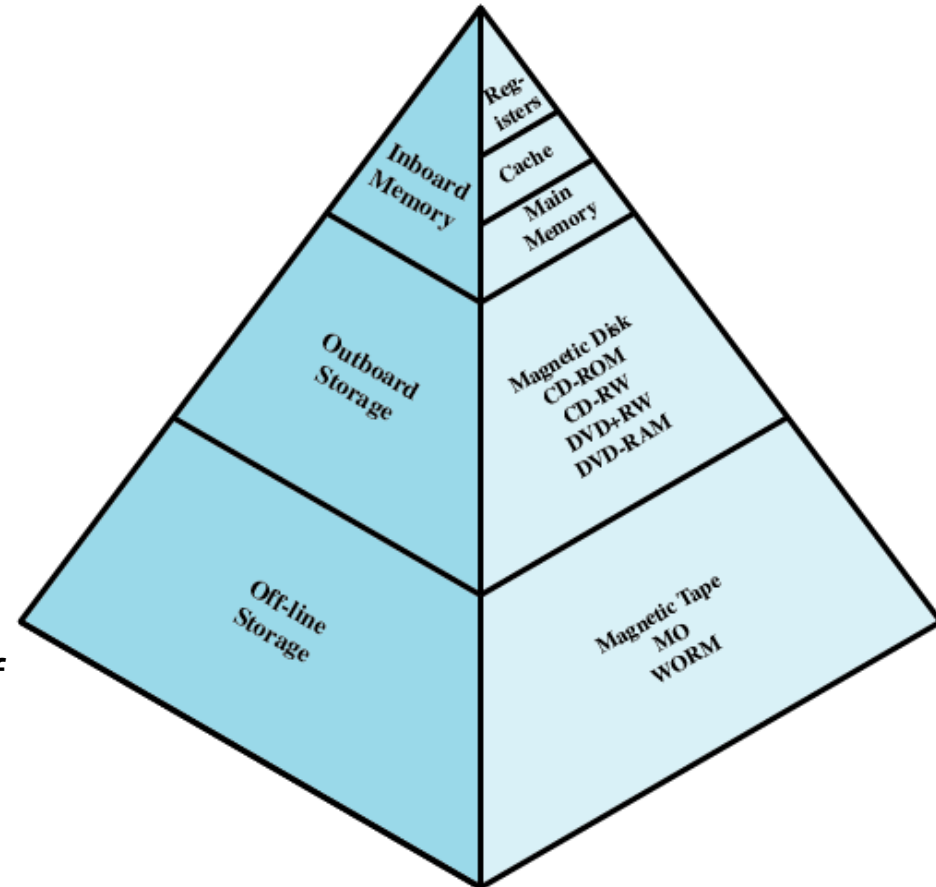
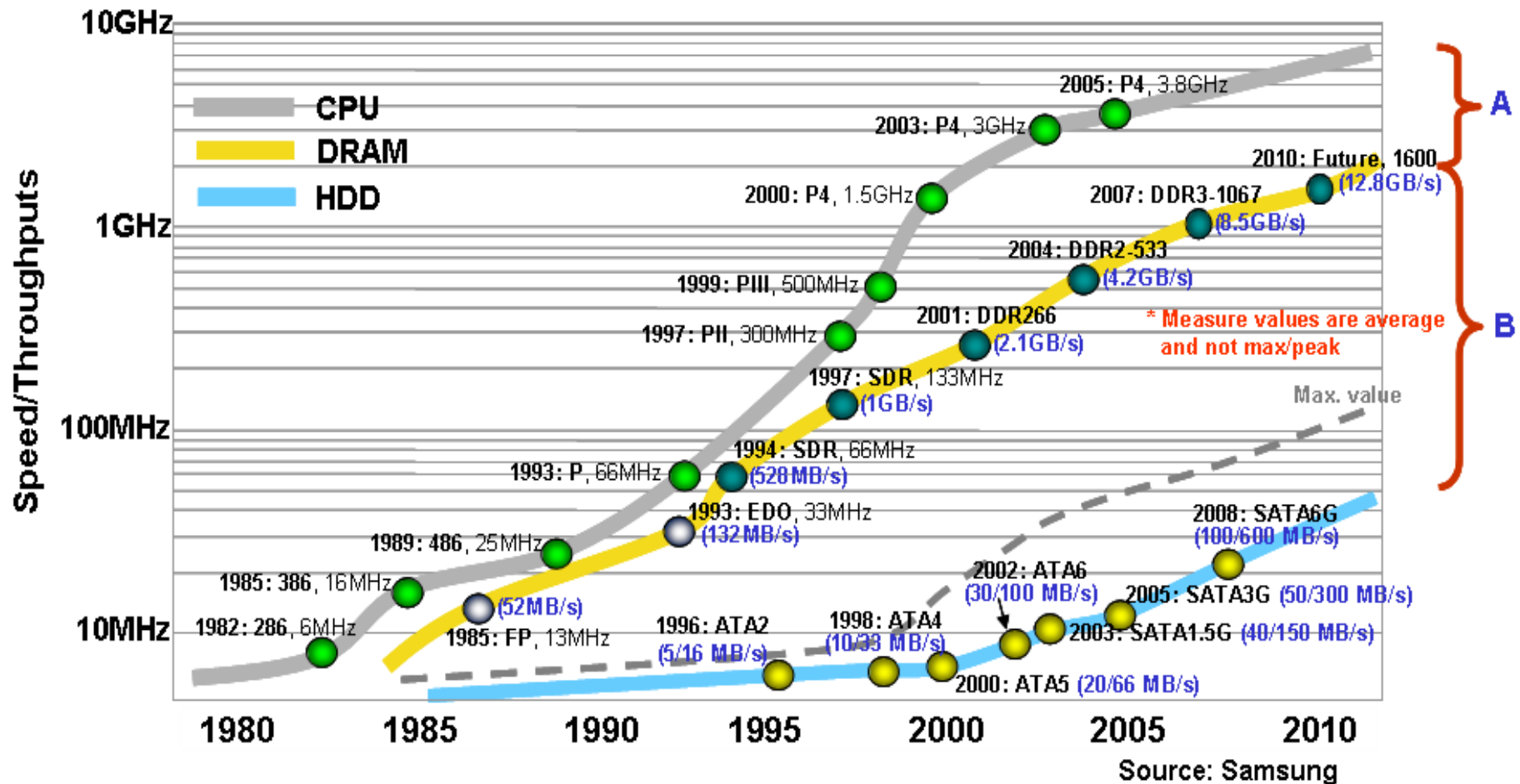


Figure 1.14 The Memory Hierarchy

H/W Speed-up Trends



Cache Memory

- The rate at which the processor can execute instructions is limited by the memory cycle time:
 - The solution is to exploit the principle of **locality** by providing a small, fast memory between the processor and main memory, namely the **cache**

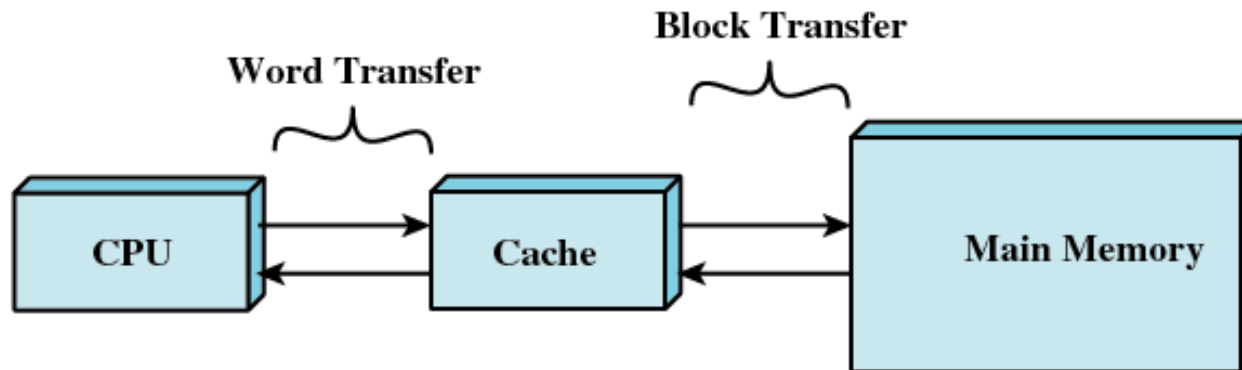
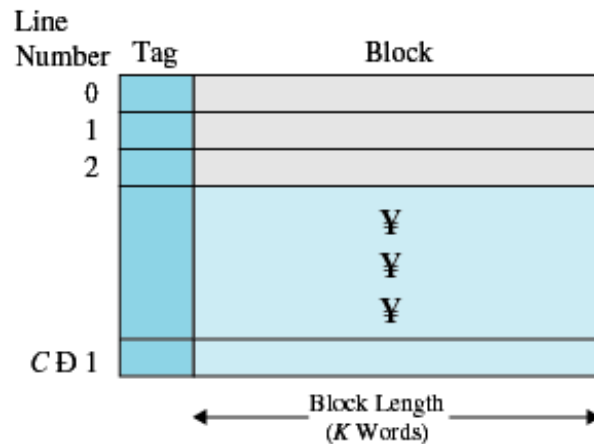
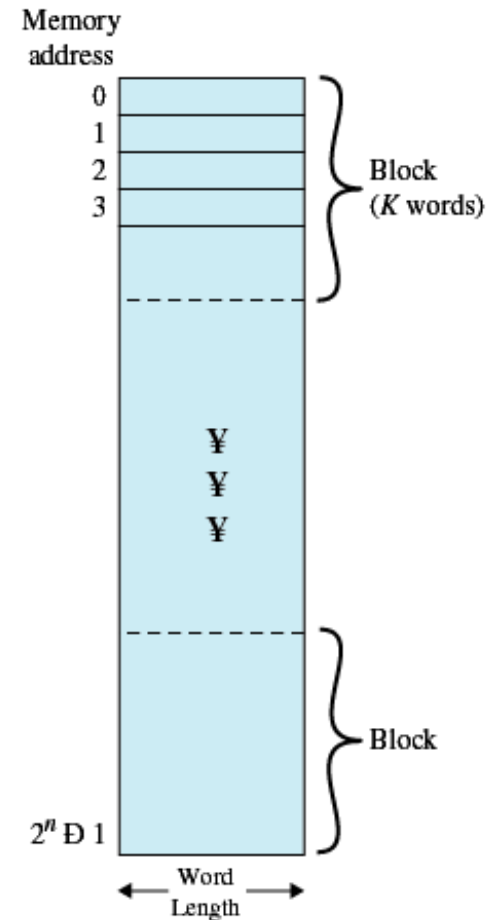


Figure 1.16 Cache and Main Memory

Cache/Main Memory System



(a) Cache



(b) Main memory

Figure 1.17 Cache/Main-Memory Structure

Cache Memory

- Contains a copy of a portion of main memory
- Processor first checks cache
- If not found in cache, the block of memory containing the needed information is moved to the cache and delivered to the processor

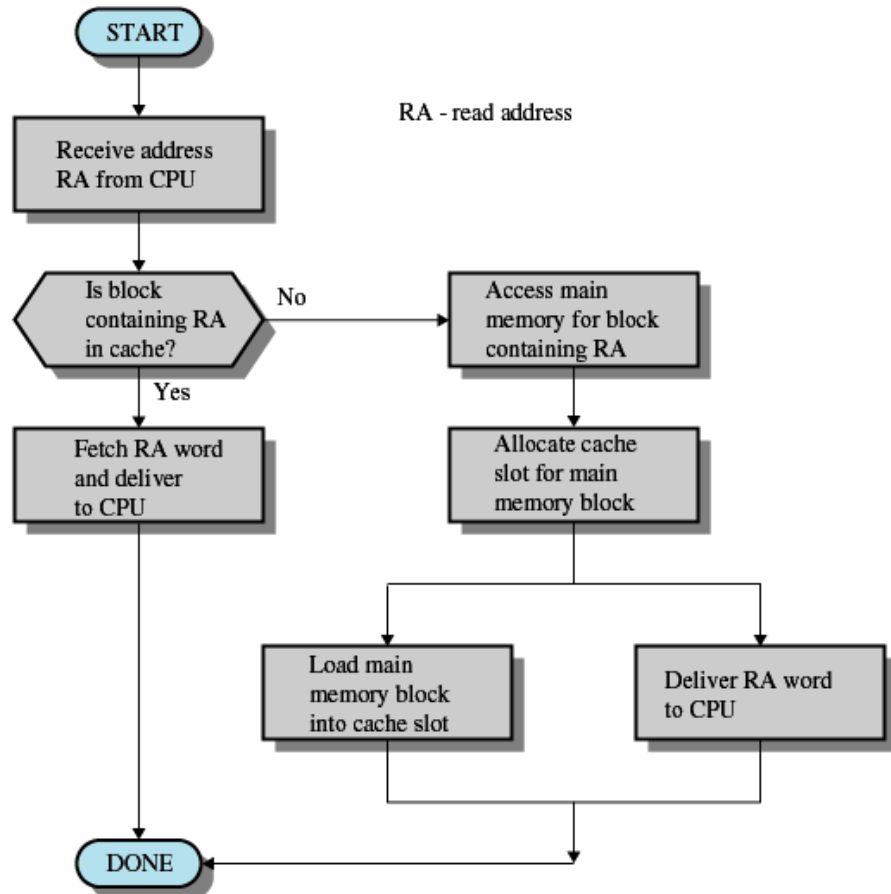
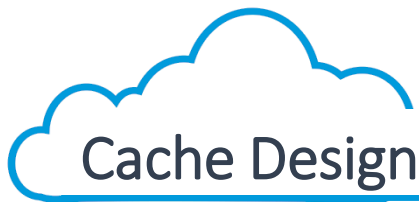


Figure 1.18 Cache Read Operation



Cache Design

- Cache size: Small caches have a significant impact on performance
- Block size
 - The unit of data exchanged between cache and main memory
 - Larger block size increases hit ratio until probability of using newly fetched data becomes less than that of reusing data that have to be moved out of cache
- **Mapping function**: Determines which cache location the block will occupy
 - Direct mapping, Associative mapping, Set-associative mapping
- Replacement algorithm
 - Determines which block to replace
 - Least-Recently-Used (LRU) algorithm
- Write policy: When the memory write operation takes place
 - Can occur every time block is updated (**write-through**)
 - Can occur only when block is replaced (**writeback**)
 - Minimizes memory write operations
 - Leaves main memory in an obsolete state
 - **Cache coherence problem** in multicore or multiprocessor system



(Optional) Special Instructions for Cache Bypass

- ‘Non-Temporal’ LOAD/STORE instructions
 - Provides hint to the memory system that caching is not useful for this data, or
 - Forcefully bypass cache memory
- Why do we use NT instructions?
 - Some data are accessed sequentially
 - E.g., array traversal, DB table scan, Multimedia file access
 - LRU based replacement scheme evicts those data that will be accessed sooner than other data → Bad performance
 - Sequentially accessed large amount of data ‘floods’ the entire cache: called, *sequential flooding*
 - Those data are better NOT to be stored in cache → Cache bypassing required

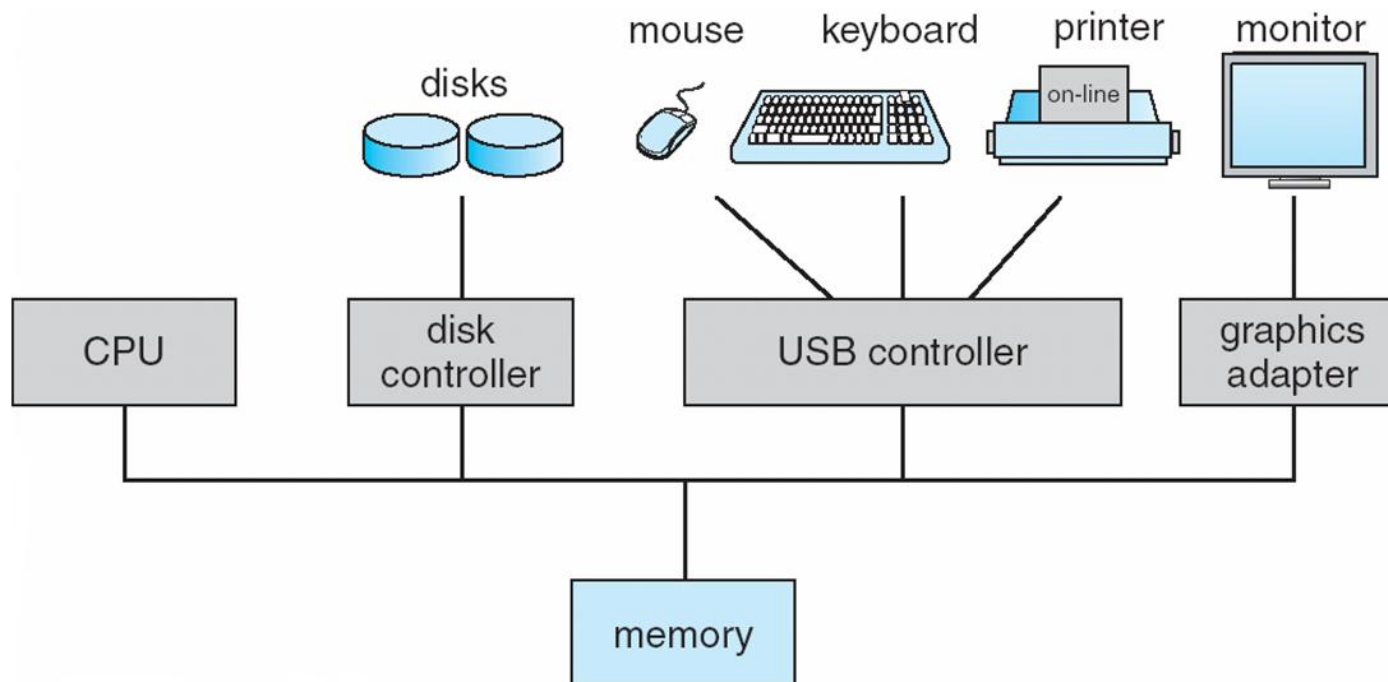


- A portion of main memory used as a buffer to temporarily hold data for the disk
 - The data are retrieved rapidly from the software cache instead of slowly from disk
- Disk writes are clustered
- Some data written out may be referenced again



Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles



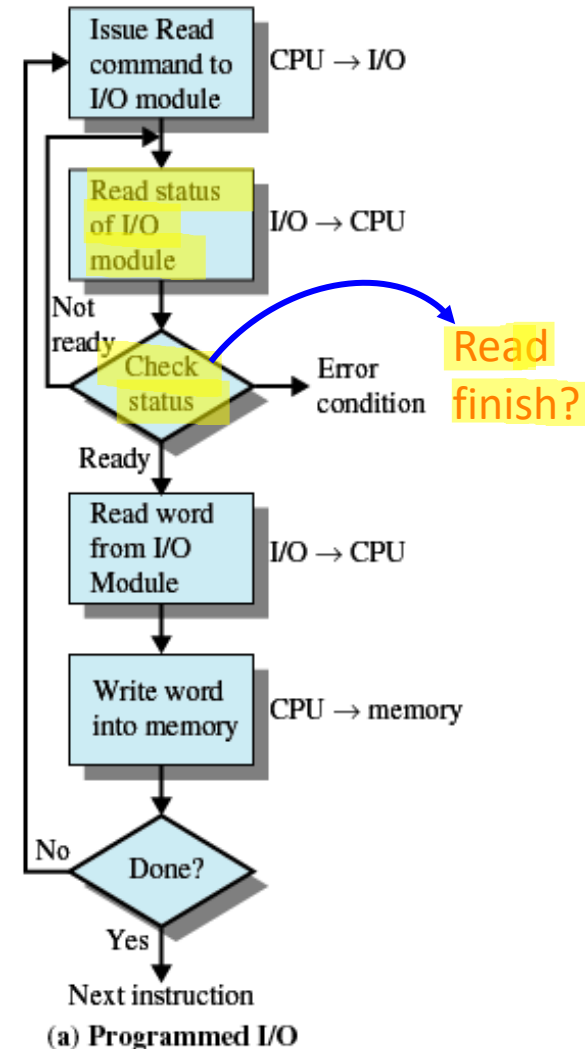


Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
 - I/O is from the device to local buffer of controller
 - I/O device cannot directly access main memory!
- Device controller informs CPU that it has finished its operation by causing an *interrupt*

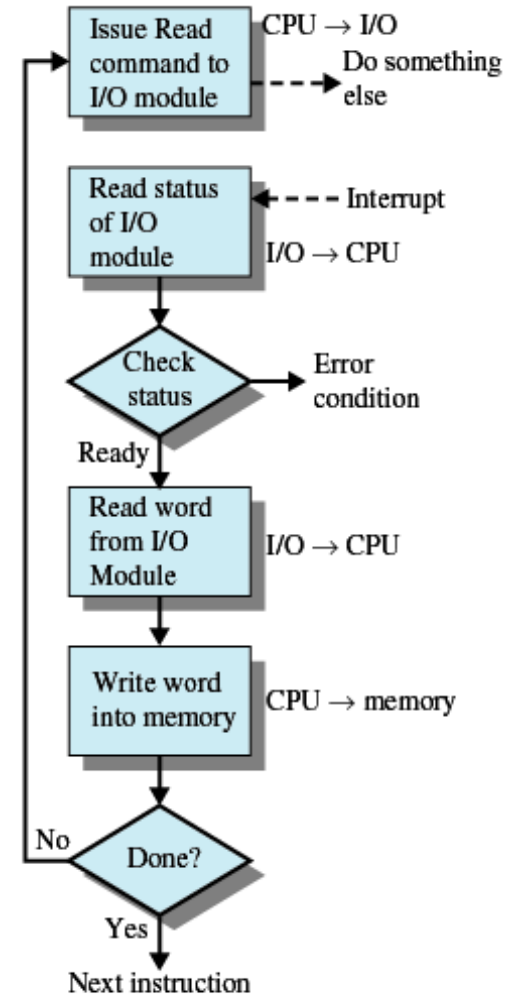
Programmed I/O

- I/O module performs the action, not the processor
- Sets appropriate bits in the I/O status register
- No interrupts occur
- Processor checks status until operation is complete



Interrupt-Driven I/O

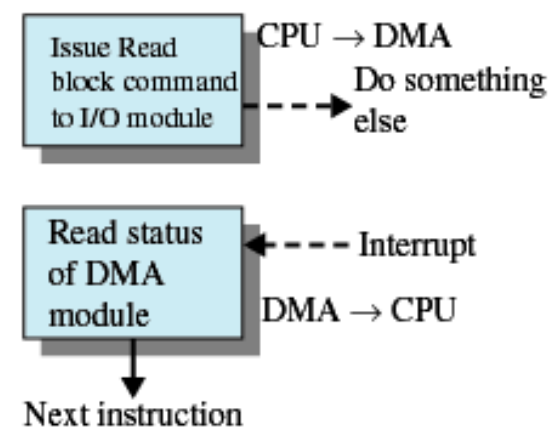
- Processor is interrupted when I/O module ready to exchange data
- Processor saves context of program executing and begins executing interrupt-handler
- No needless waiting
- Still consumes a lot of processor time because every word read or written passes through the processor



(b) Interrupt-driven I/O

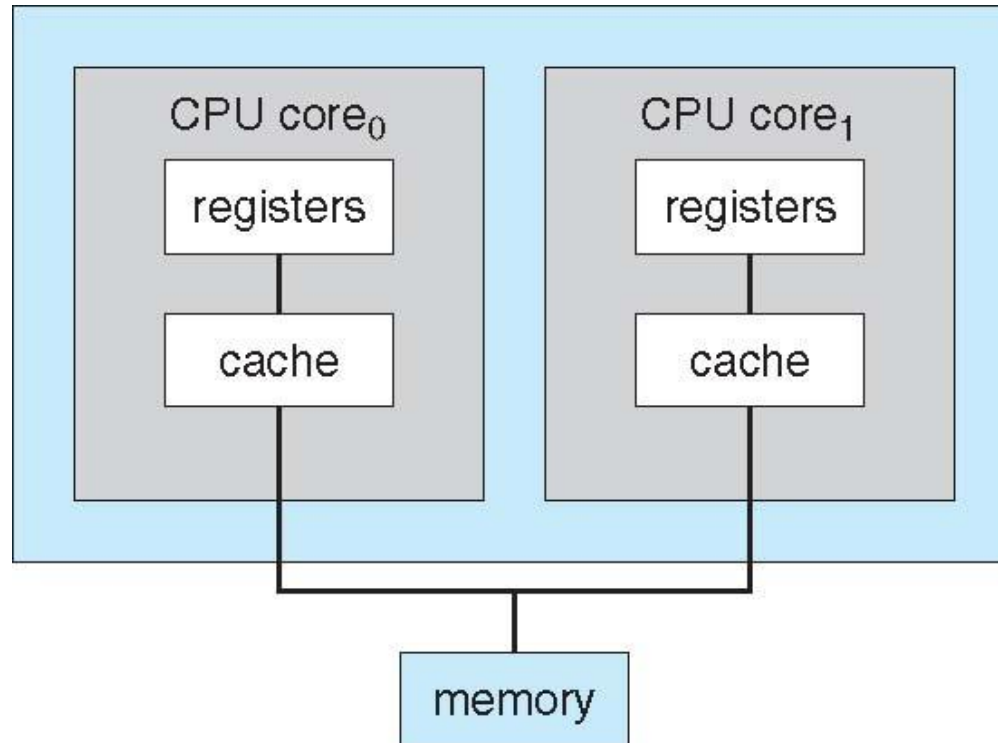
Direct Memory Access

- I/O exchanges occur directly with memory
- Processor grants I/O module authority to read from or write to memory
- Relieves the processor responsibility for the exchange
- Transfers a block of data directly to or from memory
- An interrupt is sent when the transfer is complete
- Processor continues with other work



(c) Direct memory access

A Dual-Core Design





Computer Startup

- **bootstrap program** is loaded at power-up or reboot
 - Typically stored in ROM or EPROM, generally known as **firmware**
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution